MSP Runway Tracker — Dynamic LED Architecture

System Architecture Specification: Real-Time LED Palettes & Brightness across Wi-Fi and BLE

ARCHITECTURE RFC | WI-FI + BLE IN-BAND | NVS FLASH PERSISTENCE | HARDWARE POWER CLAMPING

1. Executive Summary & Design Goals

Currently, LED visual parameters (rollout colors, takeoff afterburner hues, threshold markers, and master brightness) are compiled into the firmware as constants. This architectural document outlines a production design for updating these parameters **on the fly at any time** without re-compiling or re-flashing the ESP32-S3.

Core Engineering Requirements

- **Unified Transport Agnostic:** Works identically whether operating on the **Home setup** (Wi-Fi REST from Raspberry Pi) or the **Office setup** (Bluetooth Low Energy Nordic UART from PC).
- **Zero Runtime Lag:** Eliminates continuous JSON re-parsing on the ESP32 to preserve full 30+ FPS animation smoothness.
- **Strict Electrical Safety:** Protects against software-induced power spikes (e.g. user requesting 100% white on all 84 LEDs).
- **Cold-Boot Memory:** Remembers the user's custom theme across power cycles using non-volatile flash storage.

2. The Core Data Structure: `LedThemeConfig`

All visual configurations are encapsulated in a compact, Plain-Old-Data (POD) struct (~26 bytes) in ESP32 RAM:

```
struct LedThemeConfig {
    uint8_t  master_brightness;    // 0–255 (Soft user setting; clamped to safe ceiling)

    // 24-bit RGB values (0xRRGGBB)
    uint32_t landing_head_rgb;     // Touchdown strobe head (Default: 0xFFFFFF Crisp White)
    uint32_t landing_tail_rgb;     // Rollout glidepath tail (Default: 0x00FF88 Emerald Green)

    uint32_t takeoff_head_rgb;     // Acceleration roll head (Default: 0xFFE0A0 Warm White)
    uint32_t takeoff_tail_rgb;     // Afterburner flame tail (Default: 0xFF4500 Amber/Orange)

    uint32_t idle_threshold_rgb;   // Inactive runway threshold (Default: 0x002200 Dim Green)
    uint32_t conflict_warn_rgb;    // Intersecting runway alert (Default: 0xFF0000 Flashing Red)

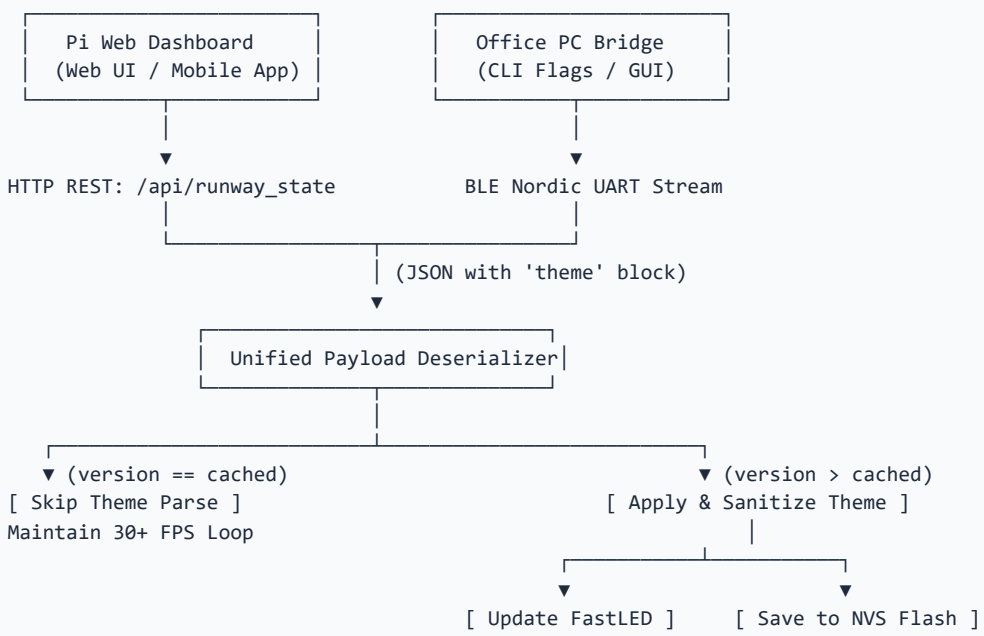
    uint16_t version;              // Monotonic version counter for dirty-checking
};
```

Why 24-bit Hex Integers?

- Matches FastLED's native `CRGB(0xRRGGBB)` constructor directly with zero translation cost.
- Easily serializes to and from standard hex color strings (e.g., `"#00FF88"` or `0x00FF88`) across web dashboards and mobile clients.

3. Unified Transport: In-Band Telemetry Embedding

Rather than creating separate REST endpoints or custom BLE services, theme data is delivered **in-band** inside the existing telemetry payload under an optional `theme` key:



Wire Format Example

```
{
  "timestamp": 1727845600,
  "status": "ACTIVE",
  "active_operations": [ ... ],
  "theme": {
    "version": 4,
    "brightness": 28,
    "landing_tail": "0x00E5FF",
    "takeoff_tail": "0xFF3300"
  }
}
```

Change Detection & Zero-Lag Parsing

- On each telemetry update, the ESP32 checks: `if (doc["theme"]["version"] == current_theme.version) return;`
- If the version is identical, the parser exits immediately, consuming negligible CPU cycles.
- If the version has incremented, the new values are unpacked, validated against safety limits, applied to FastLED, and written to non-volatile memory.

4. Platform Handling: Wi-Fi vs. BLE

Feature	Home Setup (Wi-Fi + Raspberry Pi)	Office Setup (BLE + PC Bridge)
Source of Settings	Stored on Raspberry Pi (`led_theme.json`)	Stored on Office PC (`office/settings.json`)
User Interface	Sliders & color pickers on Pi Web Dashboard (`http://msp-tracker.local:8080/settings`)	CLI flags (`bridge.py --brightness 24 --theme sunset`) or system tray app
Delivery Path	Injected into `/api/runway_state` HTTP response	Streamed across Nordic UART Service (NUS) RX
Latency	Updates on next 1.5-second polling cycle	Near-instantaneous (< 100 ms)

5. Hardware Safety Guardrails (The "Sanitizer")

> **CRITICAL: Electrical Protection**
> Allowing dynamic color and brightness configuration introduces the hazard of an accidental over-current command (e.g. setting 100% full-white on all 84 WS2812Bs pulls up to **4.2 Amperes**). The firmware must enforce hard safety clamping in code.

Layer 1: Software Brightness Ceiling

The incoming brightness value is strictly clamped before touching the driver:

```
#define HARDWARE_MAX_SAFE_BRIGHTNESS  48  // Limits worst-case consumption to ~0.8A
uint8_t target_brightness = doc["theme"]["brightness"] | current_theme.master_brightness;
current_theme.master_brightness = constrain(target_brightness, 5, HARDWARE_MAX_SAFE_BRIGHTNESS);
FastLED.setBrightness(current_theme.master_brightness);
```

Layer 2: FastLED Dynamic Power Management

FastLED features an integrated milliwatt power estimator. If simultaneous crosswind departures and arrivals illuminate multiple runways with high-draw colors, FastLED dynamically scales back luminance:

```
// Clamp overall power draw to 5V @ 1200mA (1.2 Amperes)
FastLED.setMaxPowerInVoltsAndMilliamps(5, 1200);
```

6. Non-Volatile Storage (NVS) Lifecycle

The ESP32 uses its internal Flash Non-Volatile Storage (`Preferences.h`) to retain customizations across power cycles:

1. **Boot Initialization:** During `setup()`, the firmware loads saved theme keys from NVS before network or BLE initialization begins. The board immediately boots with the user's preferred colors.
2. **Flash Wear Protection:** NVS writes only occur when the version number increases, guaranteeing that continuous telemetry streaming never creates flash wear.

7. Future Implementation Checklist

- [ ] **ESP32:** Implement `theme_manager.h` / `theme_manager.cpp` with NVS loader and safety clamps.
- [ ] **ESP32:** Hook `applyThemeIfPresent(doc["theme"])` into `network_client.cpp` and `ble_client.cpp`.
- [ ] **Raspberry Pi:** Add persistent `led_theme.json` and inject the `"theme"` block into `/api/runway_state`.
- [ ] **Raspberry Pi Web UI:** Add a settings tab to the web dashboard with brightness sliders and color pickers.
- [ ] **Office Bridge:** Add `--theme` and `--brightness` parameters to `office/bridge.py`.